Numbers to Know

快取（Caching）

現在的記憶體快取容量和效能都遠超過過去。Redis 不再只是 32GB～64GB 規模的小型快取，而是可以輕鬆支撐數百 GB 甚至 TB 級的資料，單次讀取 latency 低於 1 毫秒，每秒處理量更可達數十萬次操作。這樣的規模改變了設計思路：很多時候與其絞盡腦汁設計複雜的部分快取策略，不如乾脆「把所有資料都放進快取」。

關鍵數字

- 記憶體容量：最高可達 1TB（甚至更多的特殊配置）
- latency：同區域讀取 < 1ms；跨區寫入 1–2ms
- 吞吐量：單節點可支撐 100k 以上讀取/秒，寫入達數十萬次/秒

什麼情況下需要 Sharding

- 資料集逼近 1TB
- 長期吞吐量超過 100k ops/sec
- 讀取 latency 需要穩定在 0.5ms 以下

現在真正的瓶頸多半是「每秒操作數（ops/sec）」或「網路頻寬」，而不是記憶體大小。這跟幾年前的狀況完全相反。

> **什麼是 ops/sec？** ops/sec 不是一個固定的硬體規格（不像 RAM 就是 256GB），而是硬體 + 軟體 + 工作負載共同決定的結果。根據上下文不同，指的東西不一樣：
>
> - **DB 層面**：每秒能完成多少次 read/write query
> - **Application 層面**：每秒能處理多少個 API request
> - **Cache 層面**：Redis 每秒能回應多少次 GET/SET
>
> 同一台機器跑簡單的 key-value lookup 可能有 100k ops/sec，跑複雜的 JOIN query 可能只有 1k ops/sec。

buildmoat.org

資料庫（Databases）

現代資料庫的處理能力常常讓人意外。單一 PostgreSQL 或 MySQL 實例就能處理數十 TB 的資料，並保持毫秒級 latency。單機寫入量也能達到每秒數萬筆交易。很多時候限制因素反而不是效能，而是備份、跨區域複寫等營運考量。

關鍵數字

- 儲存：64TiB，Aurora 可達 128TiB
- latency：快取讀取 1–5ms；磁碟讀取 5–30ms；commit latency 5–15ms
- 吞吐量：Aurora/RDS 單節點讀取可達 50k TPS；寫入 10–20k TPS
- 連線數：5,000–20,000

什麼情況下需要 Sharding

- 資料規模逼近 50TiB
- 寫入吞吐長期超過 10k TPS
- 未快取的讀取要求 <5ms
- 需要跨區域佈署
- 備份時間過長或不切實際

很多人在只有 500GB 或 1～2TB 的時候就急著談 sharding。其實經過良好調校的單一資料庫，已經能支撐數百萬甚至上千萬用戶。建議先算清楚，再談 sharding。

> **「良好調校」具體指什麼？** Sharding 之前有很多更簡單的優化手段：
>
> - **慢查詢** → 加 index / 優化 SQL
> - **讀取瓶頸** → 加 cache → 加 read replica
> - **寫入瓶頸** → 調 DB 參數 → 升級硬體
>
> Sharding 主要解決的是**寫入擴展**問題（因為寫入只能走 primary，replica 幫不上忙）。但它會大幅增加系統複雜度（跨 shard 查詢、分散式交易、資料遷移等），所以是上述手段都用盡後的最後手段。

應用伺服器（Application Servers）

伺服器資源比過去豐富許多，讓許多傳統設計模式已經不再受限。現代伺服器可以同時處理十萬以上的連線，CPU、記憶體、頻寬都極為充足。通常的瓶頸是 CPU，而不

buildmoat.org

是記憶體或連線數。

關鍵數字

- 連線數：單機 100k+
- CPU：8–64 核心
- 記憶體：64–512GB（最高可達 2TB）
- 網路：25Gbps
- 啟動時間：Container apps 30–60 秒

什麼情況下需要 Sharding

- CPU 長期使用率超過 70–80%
- Latency 無法符合 SLA
- 記憶體使用率長期超過 70–80%
- 頻寬逼近 20Gbps

雖然 stateless 服務設計對擴展很重要，但別忽略單機記憶體已經很大，可以直接用來做快取、計算或 session 管理。通常第一個瓶頸是 CPU，而不是 RAM。

訊息佇列（Message Queues）

訊息佇列已經演進成高效能的數據高速公路。以 Kafka 為例，單一 broker 每秒可處理百萬級訊息，latency 僅需數毫秒，還能保存數週甚至數月的資料。這讓它的應用範圍遠超過傳統的 async 程序。

關鍵數字

- Throughput：每 broker 可達 1M 訊息/秒
- Latency：1–5ms（同區域）
- Message Size：1KB–10MB
- Storage：單 broker 可達 50TB
- Retention：數週到數月

什麼情況下需要 Sharding

buildmoat.org

- Throughput 逼近 800k messages/sec
- Partition Count 逼近 200k
- Consumer Lag 持續增加
- Cross-Region Replication

由於 latency 能穩定在 5ms 以內，訊息佇列甚至可以直接用在 Synchronous API 流程中，不再只能用於 async 背景處理。再加上幾乎無限的保存能力，使得它們能取代許多傳統專用系統，成為整個資料流的骨幹。

面試中如何使用這些數字？

你不需要精準記得每個數字，但要有基本的量級直覺：

- **Latency 順序**：memory << disk << network
- **單機容量**：一台機器大概能撐多少 QPS、多少 memory
- **資料量級感覺**：1KB / 1MB / 1GB 的差異

講的時候用「大概量級 + 推理」的方式即可，例如：

- 「假設一台機器有幾十 GB memory，大概可以放幾億個這種 object」
- 「大概超過多少 TPS 才需要考慮 database sharding」

這些數字最大的價值是幫你**避免 over-engineering**。最常犯的錯誤就是在只有幾 TB 資料、幾千 QPS 時就急著說要 sharding、要微服務。面試官會很欣賞能根據數據做出合理判斷、而不是盲目套用 fancy words 的候選人，尤其在 senior+ 的面試中。

buildmoat.org

速查表（Cheat Sheet）

Components	Key Metrics	Scale Triggers
快取 (Caching)	- Latency：約 1 毫秒 - 每秒 100k 以上操作 - 記憶體上限約 1TB	- hit rate < 80% - latency > 1ms - 記憶體使用率 > 80% - churn/thrashing
資料庫 (Databases)	- 每秒最高 50k TPS - cached read latency < 5ms - 儲存容量 64TiB+	- 寫入吞吐量 > 10k TPS - uncached read latency > 5ms - 需要跨區域分佈
應用伺服器 (App Servers)	- 支援 100k 以上 concurrent 連線	- CPU 使用率 > 70% - Response latency 超過 SLA

Components	Key Metrics	Scale Triggers
	- 8–64 核心 @ 2–4GHz - 64–512GB RAM（可至 2TB）	- 連線數逼近 10 萬/台 - 記憶體使用率 > 80%
訊息佇列 (Message Queues)	- 每 broker 每秒可處理 100 萬訊息 - 端到端 latency < 5ms - 儲存容量可達 50TB	- Throughput 逼近 800k 訊息/秒 - 每集群分區數 ~200 k - Consumer lag 持續增長

自我測驗

Q1：配對題 — 將指標與數量級配對：(1) Redis 單節點讀取延遲 (2) Kafka 單 broker 吞吐量 (3) PostgreSQL cached read (4) Aurora/RDS 單節點寫入 / (a) ~1M/sec (b) <1ms (c) 10-20k TPS (d) 1-5ms

> - 1 → (b)：Redis 讀取延遲 < 1ms
> - 2 → (a)：Kafka 吞吐量 ~1M msgs/sec per broker
> - 3 → (d)：PostgreSQL cached read 1-5ms
> - 4 → (c)：Aurora/RDS 寫入 10-20k TPS

Q2：一個調校良好的單一資料庫可以支撐多少用戶？什麼時候才真的需要 sharding？

> 一個調校良好的單一資料庫可以支撐數**百萬甚至上千萬用戶**。不要在只有 500GB-2TB 時就急著談 sharding。
>
> 需要 sharding 的時機：資料規模逼近 50TiB、寫入吞吐長期超過 10k TPS、未快取的讀取要求 <5ms、或需要跨區域佈署。

Q3：現代系統中，真正的瓶頸通常是什麼？記憶體大小嗎？

> 現代系統中真正的瓶頸多半是**每秒操作數（ops/sec）或網路頻寬**，而不是記憶體大小。這跟幾年前的狀況完全相反。

buildnoat.org